

VILNIUS UNIVERSITY
FACULTY OF MATHEMATICS AND INFORMATICS
SOFTWARE ENGINEERING STUDY PROGRAMME

Software Systems Design Project

TrafiK

Augustinas Pletkus, Edvinas Ryliškis, Audrius Duoblys
Supervisor: Vasilij Savin

Vilnius
2026

Table of contents

1. Introduction ... 3
    1.1 Overview and high concept ... 3
    1.2 Target audience ... 3
    1.3 Business problem ... 3
    1.4 Product scope ... 3
    1.5 Data sources ... 3
2. Feature list and business requirements ... 5
    2.1 Business requirements (MoSCoW priority) ... 5
        2.1.1 Must have requirements ... 5
        2.1.2 Should have requirements ... 6
        2.1.3 Could have requirements ... 6
        2.1.4 Won’t have requirements ... 7
3. Business flows ... 8
    3.1 BF-01 Journey planning ... 8
    3.2 BF-02 Starting in-ride assistance ... 9
    3.3 BF-03 In-ride tracking, alerts and replanning ... 10
    3.4 BF-04 Data processing and route-change notices ... 11
    3.5 BF-05 Community reports ... 12
    3.6 BF-06 Ticket purchase and activation ... 13
4. Non-functional requirements ... 15
    4.1 Performance and data ... 15
    4.2 In-ride assistance ... 15
    4.3 Usability and accessibility ... 15
    4.4 Device and platforms ... 16
    4.5 Security, privacy and reliability ... 16
5. High-level architecture ... 17
    5.1 Package diagram ... 17
    5.2 Class diagram: in-ride assistance ... 18
    5.3 Class diagram: server ... 18
    5.4 Journey planning ... 20
6. UI/UX wireframes ... 21
    6.1 Overview ... 21
    6.2 Conventions ... 21
    6.3 Screens ... 21
    6.4 Example scenario ... 22
    6.5 UI decisions ... 22
7. Data model ... 23

1. Introduction

1.1 Overview and high concept

TrafiK is a Vilnius public transport mobile app combining A-to-B routing, live vehicle tracking, departure boards, and ticketing with real-time in-ride assistance. Unlike standard transit planners that stop guiding users once they board, TrafiK identifies the passenger's active vehicle (manually or via GPS), tracks the ride live, alerts them before their exit stop, warns of delayed transfers, and dynamically suggests faster connections.

1.2 Target audience

The app is designed for two groups of users:

- **Vilnius residents** who use public transport daily for commuting to work or study and want fast, reliable departure information and convenient ticketing.
- **Visitors and tourists** who do not know the city network and need clear guidance on which vehicle to take, where to transfer and where to get off.

1.3 Business problem

Existing Vilnius transit apps only plan routes before departure. Once on board, passengers must manually track stops, delays, and transfer risks, often resulting in missed connections or wrong exits. Furthermore, real-time network disruptions are fragmented: ticket inspection reports are only available on an external site (raid.lt), while vehicle breakdowns and lost items lack any centralized reporting channel.

1.4 Product scope

TrafiK delivers core transit features (GTFS route planning, live vehicle map, stop departure boards, favourites) alongside two differentiators: **in-ride assistance** (live ride tracking, exit alerts, transfer risk warnings, dynamic rerouting) and **community reports** (crowdsourced alerts for inspectors, breakdowns, and lost items). The initial release prioritizes core routing and in-ride assistance; ticketing and community reports are scoped as Could have due to third-party payment/operator dependencies and too few reports until the app has enough users.

1.5 Data sources

TrafiK relies on public data provided by the city and third parties:

1. **Static schedules and network data (GTFS).** https://github.com/vilnius/transportas. Used for stops, routes, trips and timetables.
2. **Real-time vehicle positions.** https://m.stops.lt/vilnius/gps.txt. Used for the live map, delay calculation and in-ride tracking.
3. **Ticket inspector reports.** https://raid.lt/feed/inspectors. Used as an additional source for inspection reports alongside user reports.
4. **Addresses, places and walking paths.** OpenStreetMap (https://www.openstreetmap.org) through

self-hosted geocoding and walking-routing services (section 5.1).

5. **Route changes and stop closures.** Derived by diffing consecutive GTFS versions (BF-04).
6. **Ticketing and payment:** No public ticketing API exists for Vilnius public transport. In this release, TicketOperator (section 5.3) is a local mock that issues and activates tickets, and PaymentProvider uses the provider's sandbox (test) mode, so no real payments are made.

2. Feature list and business requirements

The requirements support three business goals: (G1) help passengers avoid missed transfers and missed stops by assisting them during the ride; (G2) give Vilnius residents and visitors one app for journey planning, live departures and tickets; (G3) collect passenger reports about events on the network and share them with other passengers.

2.1 Business requirements (MoSCoW priority)

2.1.1 Must have requirements

M-1 The system provides a single search field in which users can find stops, lines, addresses and places.

M-2 The system shows matching suggestions while the user types in the search field.

M-3 When the user selects a line, the system shows its stops in order and its timetable for each direction.

M-4 The system enables users to plan journeys between a starting point and a destination, including transfers between buses and trolleybuses.

M-5 For each journey search, the system shows the journey options found, sorted by arrival time, or by departure time (latest first) when the user sets a latest arrival time. Each option shows the departure and arrival time, total travel time, lines used, number of transfers and total walking time.

M-6 The system allows users to set the starting point or destination to “My location”. The system routes from the device’s exact coordinates, using nearby addresses only as display labels.

M-7 The system allows users to set the starting point or destination by placing a pin on the map, and uses the exact coordinates of the pin.

M-8 When a new journey search starts and the user is not on a known vehicle, the system uses the user’s current location as the default starting point and the current time as the default departure time.

M-9 If the device’s location is unavailable or the user has not allowed location access, the system asks the user to enter a starting point instead of using “My location”, and all functions that do not depend on the user’s location remain available.

M-10 The system plans the walking parts of a journey: from the starting point to the first stop, between stops when a transfer requires walking, and from the last stop to the destination.

M-11 The system displays buses and trolleybuses on a map using available real-time vehicle location data.

M-12 The system provides its user interface in Lithuanian and English.

M-13 The system displays upcoming departures for a selected stop, ordered by expected departure time. Each departure shows the line number, direction and scheduled time, and also the expected time or delay when real-time data is available.

M-14 The system allows users to manually select the bus or trolleybus they are on via the "I'm travelling on bus No. X" action. The system lists the vehicles currently closest to the user first, and also allows the user to enter a line number and then select the vehicle by its direction and next stop.

M-15 While the user is on a known vehicle, the system automatically uses that vehicle’s live location as the starting point for any new or updated journey, without asking the user to enter a starting point.

M-16 The system warns the user when a delay puts a planned transfer at risk.

M-17 When a planned transfer is at risk, the system recalculates the journey and offers updated options if

alternative connections are available.

M-18 The system displays upcoming stops along the user's journey and, when the vehicle leaves the stop before the user's exit stop, notifies the user to get off at the next stop.

M-19 The system allows the user to end in-ride assistance manually at any time.

2.1.2 Should have requirements

S-1 The system allows users to save stops, places, lines and journeys as favourites for quick access. Favourites are stored on the device, so that no account is needed (C-1).

S-2 The system maintains a list of recently searched stops, lines, addresses and places, stored on the device.

S-3 When the user opens the search field, the system shows recent searches and favourites as the first suggestions.

S-4 The system shows the stops nearest to the user's current location, sorted by walking distance, together with their next departures.

S-5 The system allows users to change the departure time of a journey search, or to set the latest arrival time instead.

S-6 The system automatically detects which bus or trolleybus the user is travelling on by analysing the phone's GPS location, movement speed and available vehicle location data.

S-7 The system asks users to confirm an automatically detected vehicle before activating in-ride assistance.

S-8 The system automatically ends in-ride assistance when the user's position no longer matches the vehicle's position, for example after getting off. Thresholds are defined in section 4.2.6.

S-9 The system notifies users of relevant route changes or stop closures that affect their planned or ongoing journeys, when such information is available.

S-10 The system allows users to turn each type of notification on or off, such as get-off alerts, transfer warnings and route-change notices.

S-11 The system suggests alternative connections during a ride when they save at least 3 minutes (section 4.2.4), showing the new vehicle, transfer stop, walking route and time saved.

S-12 The system shows step-by-step guidance for each part of a journey. The guidance includes the line number and direction, the stop to board at, the number of stops to travel, the transfer stop and the stop to get off at.

S-13 The system displays a persistent lock-screen widget during active rides (an iOS Live Activity or an Android ongoing notification).

2.1.3 Could have requirements

C-1 The system allows users to create an account and sign in. An account is required to purchase tickets and to submit community reports; all other functions are available without an account.

C-2 The system allows users to submit community reports about ticket inspections, broken-down vehicles and lost or found items on buses and trolleybuses.

C-3 The system displays current community reports to other users on the map and in the details of affected journeys. Reports are hidden automatically for each report type in section 4.5.3.

C-4 The system shows ticket inspection reports from the raid.lt feed alongside user reports, clearly marked with their source.

C-5 The system allows users to confirm or dismiss a community report shown to them, and hides a report

early when enough users dismiss it. The number of dismissals needed is defined in section 4.5.3.

C-6 The system enables users to purchase and activate public-transport tickets through the application.

C-7 The system displays the user's active ticket and its remaining validity, including when the device is offline.

C-8 The system allows users to request step-free journeys that use only stops and vehicles marked as wheelchair accessible, where this information is available in the GTFS data.

2.1.4 Won't have requirements

W-1 The system will not provide journey planning for public transport outside Vilnius.

W-2 The system will not provide journey planning for other transport modes, such as trains, intercity buses, taxis, bikes or scooters.

W-3 The system will not provide journey planning without an internet connection.

W-4 The system will not provide manual moderation or verification of community reports in this release.

W-5 The system will not provide password reset or server-side sign-out in this release; signing out deletes the token on the phone.

3. Business flows

This section describes the multi-step processes of TrafiK. Simple actions, such as saving a favourite or opening a departure board, are not modelled. Thresholds and limits are defined in section 4.

The diagrams use the modules of Figure 1 (section 5.1).

3.1 BF-01 Journey planning

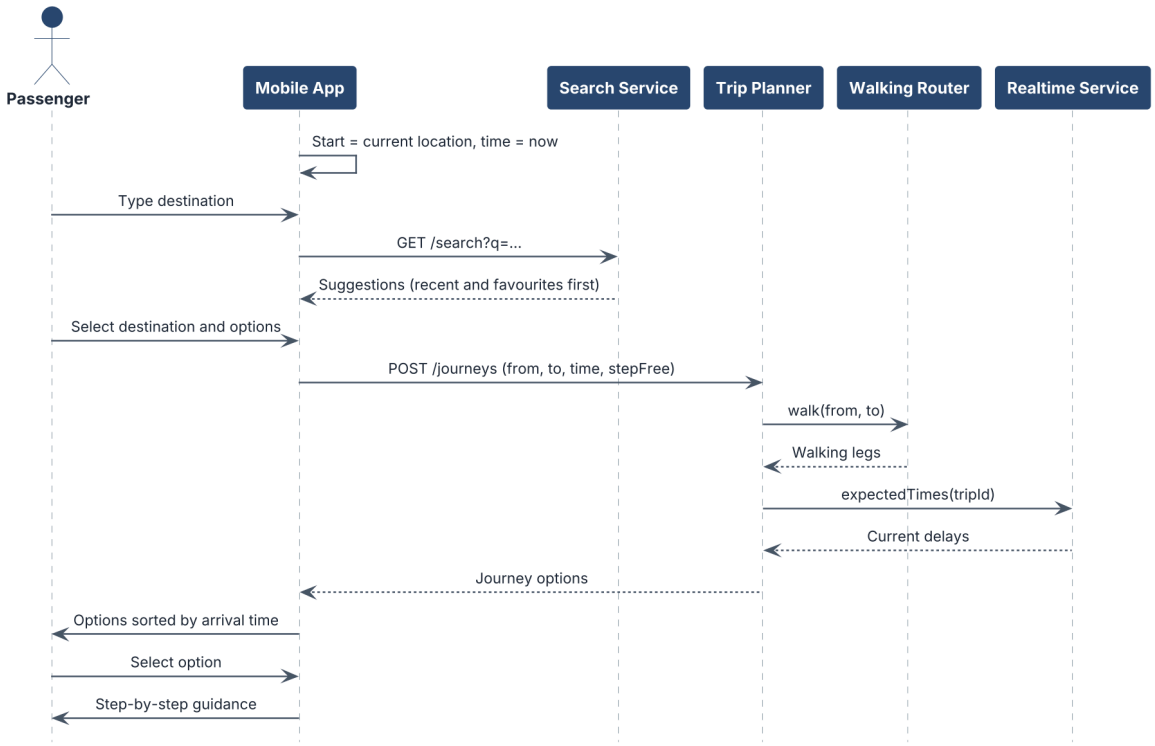
Priority: Must have. **Trigger:** The user opens the journey search. **Related requirements:** M-1 to M-10, M-15, S-2, S-3, S-5, S-12, C-8.

Main flow:

1. The app sets the starting point to the user's current location and the departure time to now.
2. As the user types a destination, the app shows matching local recent searches and favourites from device storage first, followed by stops, lines, addresses and places returned by the Search Service.
3. The user selects a destination or places a pin. Optionally, the user changes the time, sets a latest arrival time or requests a step-free journey.
4. The Trip Planner gets walking times from the Walking Router and current delays from the Realtime Service, and calculates options from the timetable using the rules in section 5.4.
5. The app shows the options sorted as defined in M-5, with departure and arrival times, lines, number of transfers and walking time.
6. The user selects an option and sees step-by-step guidance. The destination is saved to recent searches.

Alternative flows:

- **Location unavailable (step 1):** the app asks the user to enter a starting point.
- **User is on a known vehicle (step 1):** the starting point is the vehicle's live position and cannot be changed.
- **No real-time data (step 4):** options use timetable times and are marked as scheduled.
- **No results found (steps 2, 5):** the app displays an empty-state message and prompts the user to adjust the search.
- **Offline:** the app notifies the user that journey planning requires an internet connection.



Sequence diagram BF-01. Journey planning

3.2 BF-02 Starting in-ride assistance

Priority: Must have; automatic detection is Should have. **Trigger:** The user taps “I’m travelling on bus No. X”, or, while the app is open, the phone moves faster than walking speed (7 km/h, section 4.2.5). **Related requirements:** M-14, M-15, S-6, S-7, S-13.

Main flow (manual):

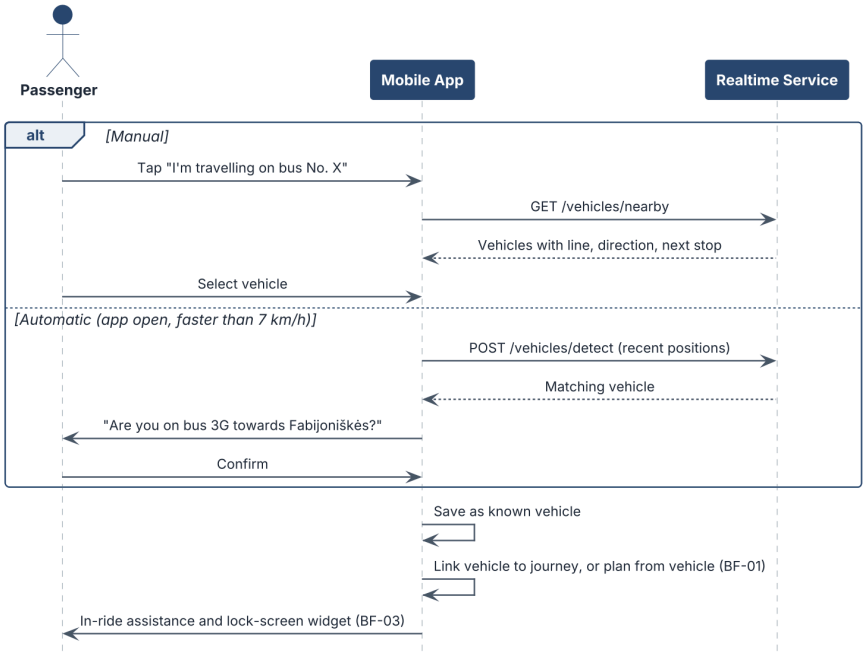
1. The app lists the vehicles closest to the user, each with its line, direction and next stop.
2. The user selects a vehicle. If it is not listed, the user enters a line number and selects the vehicle by direction and next stop.
3. The app saves it as the known vehicle.
4. If an active journey matches the line, the app links the vehicle to it; otherwise, it prompts for a destination to route from the vehicle's live position (BF-01). Skipping the destination starts unguided tracking, which shows upcoming stops without get-off or transfer alerts and ends only when the user taps End or by the automatic end (section 4.2.6).
5. In-ride assistance starts, the lock-screen widget appears and BF-03 follows.

Main flow (automatic):

1. The app sends its recent positions to the Realtime Service, which compares them with nearby vehicles.
2. When one vehicle matches clearly, the app asks, for example: “Are you on bus 3G towards Fabijoniškės?”
3. If the user confirms, the flow continues at manual step 3.

Alternative flows:

- **Location unavailable:** the user enters the line number (manual step 2).
- **No live position for the vehicle:** the app cannot confirm the vehicle or match it to a trip, so in-ride assistance does not start, and the user can choose another vehicle.
- **Active journey uses another line:** the app offers to plan a new journey from the vehicle.
- **Several vehicles match (automatic):** the app waits for more positions, then lists the candidates.
- **User answers No (automatic):** the vehicle is not proposed again during this ride.
- **App in the background:** automatic detection pauses (section 4.2.5).



Sequence diagram BF-02. Starting in-ride assistance

3.3 BF-03 In-ride tracking, alerts and replanning

Priority: Must have; the faster-alternative suggestion and the automatic end are Should have. **Trigger:** In-ride assistance starts (BF-02). **Related requirements:** M-15 to M-19, S-8, S-10 to S-13.

Main flow:

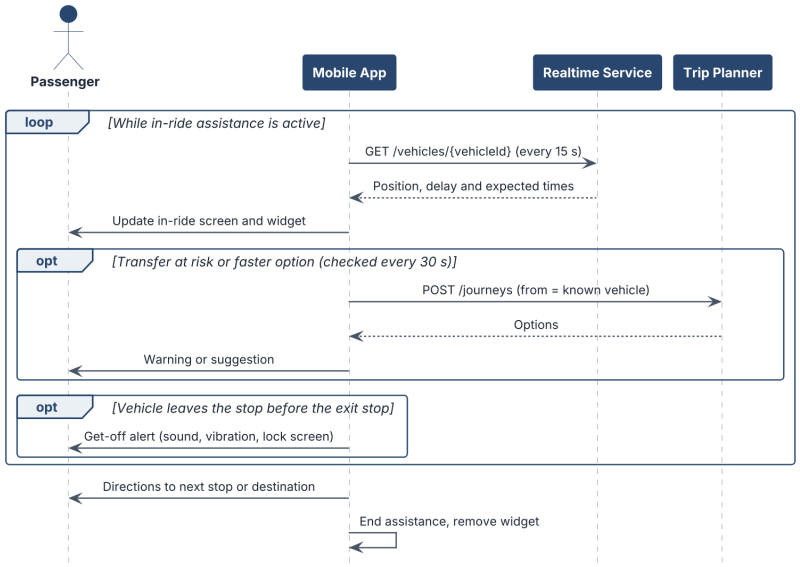
1. The app tracks ride progress via phone GPS matched against line stops (section 4.2.1).
2. The app polls the vehicle position and delays every 15 seconds to update the next stop, remaining stops, ETA and transfer status on the screen and lock-screen widget.
3. The app evaluates transfer risk every 30 seconds (section 4.2.3). If a transfer is at risk, the app alerts the user and queries the Trip Planner for reroutes from the vehicle's position.
4. If an alternative route saves at least 3 minutes (section 4.2.4), the app suggests it with transfer and walking details.
5. When the vehicle leaves the stop before the exit stop, the app sends the get-off alert with sound, vibration and a lock-screen notification.
6. At a transfer, the app shows walking directions to the next stop. When the next vehicle of the journey

arrives, the app proposes it as the known vehicle, and after the user confirms it the flow continues at BF-02, manual step 3.

7. After the last vehicle, the app shows walking directions to the destination, ends in-ride assistance and removes the widget.

Alternative flows:

- **User taps End (any step):** the flow continues at step 7 without directions.
- **Movement no longer matches the vehicle:** assistance ends automatically and the user is notified (section 4.2.6).
- **No destination (unguided tracking):** only steps 1 and 2 apply. Assistance ends only when the user taps End or by the automatic end (section 4.2.6).
- **No real-time data:** times are shown as scheduled and no transfer warnings are given.
- **No alternative found (step 3):** the warning stays, and the next departure of the planned line is shown.
- **User stays on past the exit stop:** the app offers a new journey from the vehicle.
- **Notifications refused or turned off:** alerts appear only in the app and on the widget.



Sequence diagram BF-03. In-ride tracking, alerts and replanning

3.4 BF-04 Data processing and route-change notices

Priority: Must have; route-change notices are Should have. **Trigger:** A server schedule: every 10 seconds for vehicle positions and at least once per hour for timetables. **Related requirements:** M-11, M-13, M-16, M-17, S-9; sections 4.1.2 to 4.1.4.

Main flow (vehicle positions):

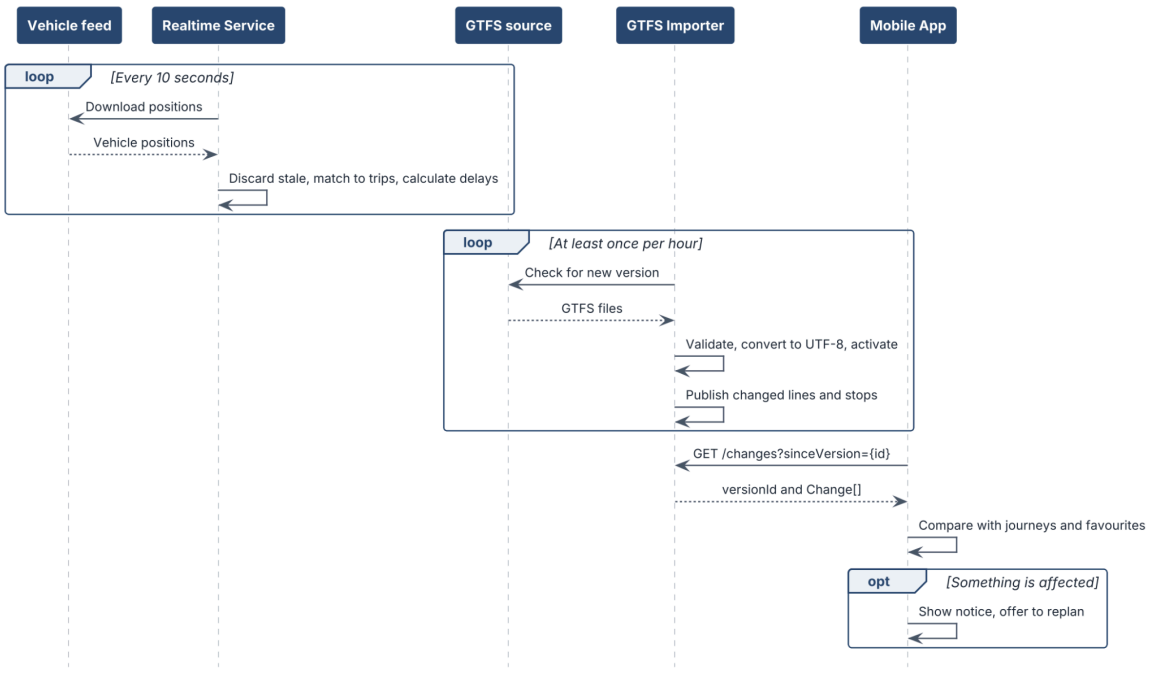
1. The Realtime Service downloads the vehicle feed and discards stale or invalid positions.
2. It matches each vehicle to a timetable trip by line, direction, position and time of day.
3. It calculates the delay and the expected times at the remaining stops, and makes them available to the other services.

Main flow (timetables and route changes):

1. The GTFS Importer checks for a new version, validates it and converts Windows-1257 text to UTF-8.
2. It activates the new version in one step and publishes a list of changed lines and stops, with the dates they apply from.
3. The app downloads the list when it starts and during in-ride updates, and compares it on the phone with the active and selected journeys and the favourites.
4. If something is affected, the app shows a notice and offers to replan (BF-01, or BF-03 during a ride).

Alternative flows:

- **No new feed data for 5 minutes:** the server raises a monitoring alert. Vehicles without a position update for 2 minutes are already shown with scheduled times (section 4.1.2).
- **Invalid GTFS data:** the previous version stays active and the team is alerted.
- **Vehicle cannot be matched to a trip:** it is shown on the map without a delay.
- **Route-change notices turned off:** changes appear only as labels on the affected items.



Sequence diagram BF-04. Data processing and route-change notices

3.5 BF-05 Community reports

Priority: Could have. **Trigger:** A user taps Report. **Related requirements:** C-1 to C-5; sections 4.5.3 and 4.5.5.

Main flow:

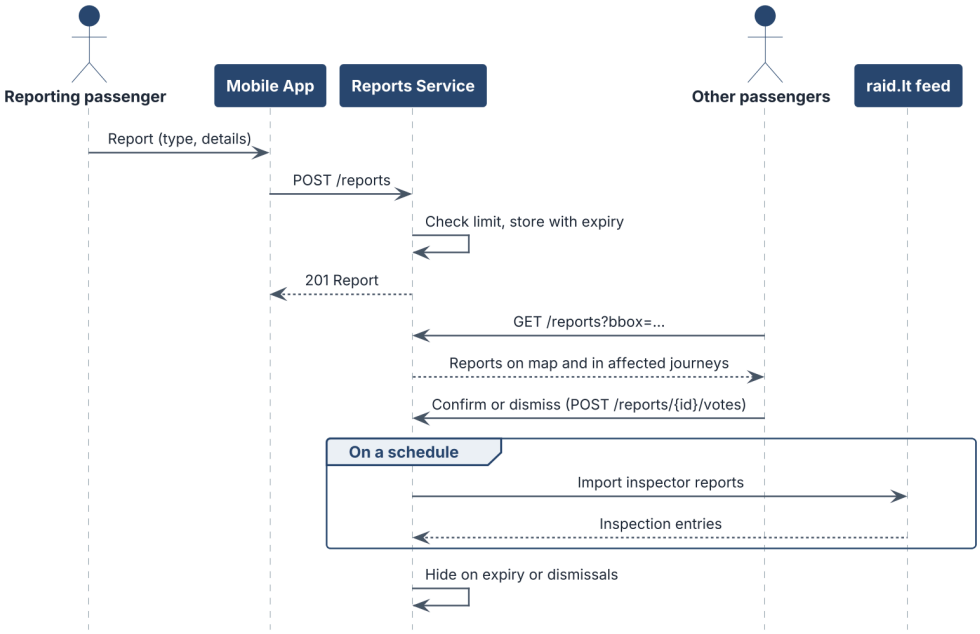
1. The user selects the type: ticket inspection, broken-down vehicle, or lost or found item.
2. The app fills in the known vehicle or, if there is none, the stop nearest to the user's location (section 7). For a lost or found item, the user adds the line, the time and a description.
3. The Reports Service checks the account's report limit and stores the report with an expiry time based on its type.
4. Other users see the report on the map and in affected journeys, without the reporter's identity, and can

confirm or dismiss it.

5. The report is hidden when it expires or when enough users dismiss it.
6. Separately, the Reports Service imports raid.lt entries as inspection reports marked with their source and a link to raid.lt.

Alternative flows:

- **Not signed in:** the app asks the user to sign in first.
- **Report limit reached:** the report is not saved.
- **Duplicate active report (matching type and stop/line/vehicle):** the report is added as a confirmation of the existing report instead of a new one.



Sequence diagram BF-05. Community reports

3.6 BF-06 Ticket purchase and activation

Priority: Could have. **Trigger:** A user selects Buy. **Precondition:** In this release, the ticketing operator and the payment provider are mocked (section 1.5). **Related requirements:** C-1, C-6, C-7; section 4.5.2.

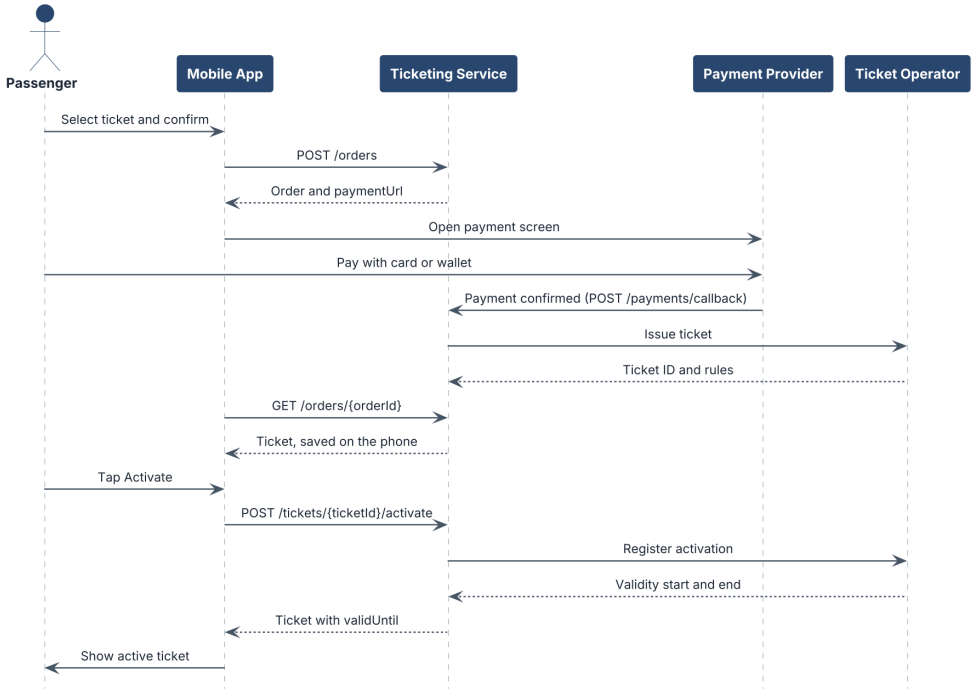
Main flow:

1. The app shows ticket types and prices from the Ticketing Service.
2. The user selects a ticket and confirms, and the Ticketing Service creates a pending order.
3. The user pays in the Payment Provider’s screen. Card data never reaches TrafiK.
4. After the payment is confirmed, the operator issues the ticket and the app stores it on the phone.
5. The user taps Activate, and the operator returns the validity period.
6. The app shows the active ticket with its remaining time, an animated element and the current time, also when offline.

Alternative flows:

- **Not signed in:** the app asks the user to sign in first.

- **Payment declined or cancelled:** the order is cancelled.
- **Ticket cannot be issued after payment:** issuing is retried, then the payment is refunded.
- **No connection when activating:** the user is asked to activate the ticket when online.



Sequence diagram BF-06. Ticket purchase and activation

4. Non-functional requirements

Times are 95th-percentile values measured on a mid-range phone over a 4G connection. Threshold values shall be stored on the server so that they can be tuned after testing without an app update.

4.1 Performance and data

1. **Response times:** Search suggestions shall appear within 1 second, journey options within 3 seconds and stop departure boards within 2 seconds.
2. **Live data:** Vehicle positions and expected times on screen shall be refreshed at least every 15 seconds. Positions older than 2 minutes shall not be shown as live, and times without real-time data shall be labelled as scheduled.
3. **Timetable updates:** The GTFS feed shall be checked for changes at least once per hour.
4. **Degraded operation:** If real-time data or address search is unavailable, the app keeps working with timetable data and stop and line search, and notifies the user that live data is unavailable.
5. **Load:** The server shall meet the response times in 4.1.1 with 5,000 users active at the same time, of whom 2,000 are in in-ride assistance.

4.2 In-ride assistance

1. **Position source:** During in-ride assistance, the user’s progress shall be tracked from the phone’s own location, sampled at least once per second, and the vehicle feed shall be used only to confirm the vehicle and update delay estimates, at least every 15 seconds. The user’s position shall never depend on vehicle feed positions, which may be up to 2 minutes old.
2. **Get-off alert:** The alert shall be delivered within 10 seconds after the vehicle leaves the stop before the exit stop, using sound, vibration and a lock-screen notification.
3. **Transfer risk:** A boarding margin of 1 minute shall be added to the walking time between stops, and the risk shall be re-checked at least every 30 seconds.
4. **Alternative connections:** A suggested alternative shall save 3 minutes, and no more than one new suggestion shall be shown every 2 minutes.
5. **Automatic detection:** Detection shall start when the phone moves faster than 7 km/h, shall run only while the app is open, and shall identify the correct vehicle in at least 90% of 20 test rides.
6. **Automatic end:** In-ride assistance shall end when, for at least 60 seconds, the user moves slower than 7 km/h while the vehicle moves faster than 15 km/h, or the user is more than 1000 m from the vehicle’s latest reported position.

4.3 Usability and accessibility

1. **Usability testing:** In tests with at least 5 first-time users, including 2 who do not know Vilnius, at least 80% shall plan a journey with a transfer and start in-ride assistance without help within 2 minutes each. The average System Usability Scale (SUS) score shall be at least 70.
2. **Glanceable in-ride screen:** The next stop and the number of stops left shall be visible on the in-ride

screen and the lock screen without any interaction.

3. **Accessibility:** The app shall meet WCAG 2.1 level AA and work with TalkBack and VoiceOver. Information shown on the map shall also be available as a list, text shall scale up to 200%, and colour shall never be the only way to show information.
4. **Language:** The app shall be available in Lithuanian and English, following the phone's language. Search shall find names typed without Lithuanian letters, for example "Zirmunai" for "Žirmūnai".

4.4 Device and platforms

1. **Battery and data:** In-ride assistance with the screen off shall use no more than 4% of the battery per 30 minutes and no more than 5 MB of mobile data per hour.
2. **Permissions:** The app shall ask for location access only when first needed, explain why, and work with "while using the app" permission. If notification permission is refused, alerts shall be shown inside the app only.
3. **Platforms:** The app shall support Android 10 or later and iOS 16.1 or later.

4.5 Security, privacy and reliability

1. **Privacy:** The system shall comply with the GDPR. Precise locations shall not be linked to user accounts, server logs containing coordinates shall be deleted within 7 days, and users shall be able to delete their account and data in the app.
2. **Security:** All communication shall use HTTPS with TLS 1.2 or later. Passwords shall be stored only as Argon2 or bcrypt hashes, and card data shall be handled only by a PCI DSS-compliant payment provider.
3. **Community reports:** Reports shall be shown without the reporter's identity, and each account shall be limited to 10 reports per hour. Reports shall be hidden after 30 minutes for ticket inspections, 60 minutes for broken-down vehicles and 7 days for lost or found items, or earlier after at least 3 dismissals that outnumber confirmations.
4. **Availability:** The server-side services shall be available 99.5% of each month.
5. **Licences:** Each data source shall be used according to its terms, with the attribution "© OpenStreetMap contributors" for map data. The public Nominatim service shall not be used for search-as-you-type, the public OpenStreetMap tile servers shall not be used for the app's map, and raid.lt entries shall be shown with their source and a link to raid.lt (C-4).

5. High-level architecture

TrafiK is a mobile app backed by a set of server-side services. The app shows the user interface and runs in-ride assistance. The server holds what all users share: timetables, live vehicle positions, journey planning, accounts, reports and tickets. Module names match those used in the business flows (section 3).

5.1 Package diagram

Figure 1 shows the system in three layers: the app, the server and external systems. Arrows show which module calls which. Dashed boxes are Could have modules.

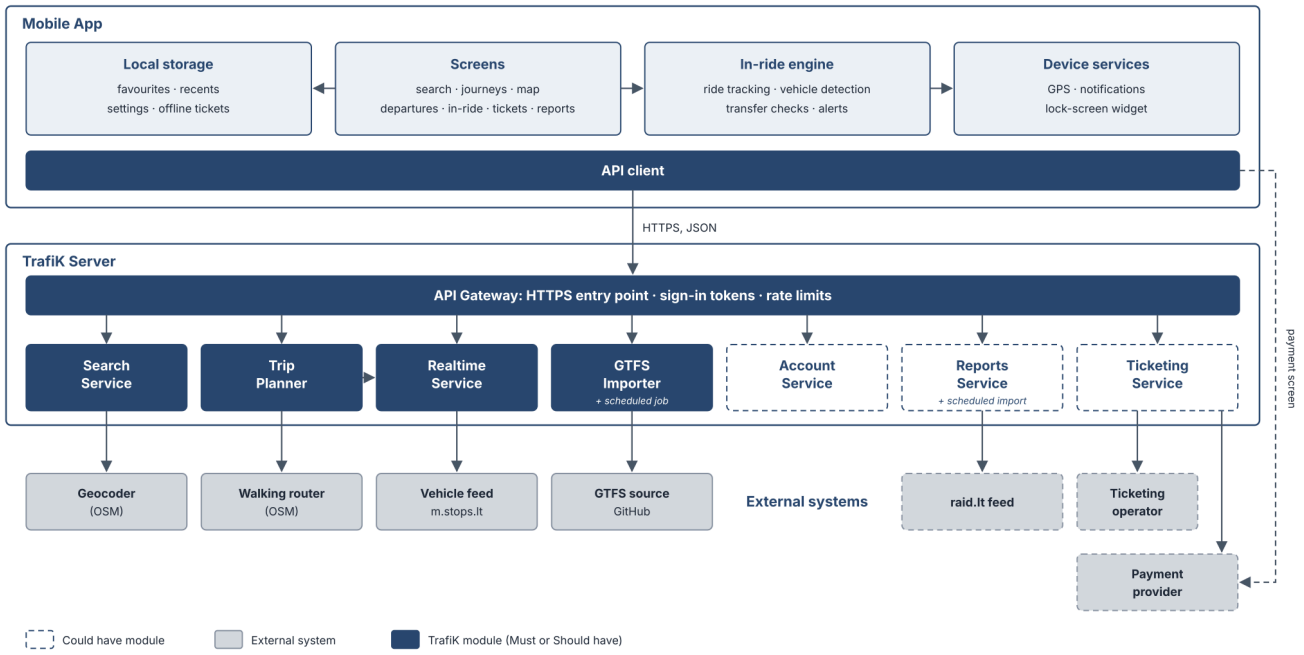


Figure 1. Modules of TrafiK and their dependencies

Key design decisions:

- **In-ride logic runs on the phone.** Tracking, get-off alerts and transfer checks use the phone's own location and run inside the app, so they keep working with the screen locked and need no push service. To keep location updates running with the screen locked, the app uses a foreground service with the location type on Android and the location background mode on iOS. Both are started while the app is open, so the "while using the app" permission is enough (section 4.4.2). The server never stores the user's journey (section 4.5.1).
- **One entry point.** The app reaches the server only through the API Gateway, which checks sign-in tokens for account functions and limits request rates. Card details go from the app straight to the Payment Provider.
- **Versioned timetable.** The GTFS Importer loads each new version in full and then makes it active in one step. Search Service, Trip Planner and Realtime Service read only the active version. The Realtime Service keeps live positions in memory and replaces them every 10 seconds.
- **Self-hosted map services.** Address search, walking routes, and map tiles use self-hosted OpenStreetMap

services (e.g., Photon, GraphHopper, OpenMapTiles) to comply with OSM usage policies (section 4.5.5).

- **Modular deployment.** Each server module is a separate package that owns its data. In the first release all modules run as one application; a module can be moved to its own service later if its load requires it.
- **Technology choice.** Programming languages, frameworks and databases are left to the implementing team. The chosen technologies shall support Android and iOS (section 4.4.3), including the lock-screen widget and location updates while the screen is locked, and shall meet the non-functional requirements (section 4).

5.2 Class diagram: in-ride assistance

Figure 2 shows the classes of the in-ride engine, the most complex module of the app. Together they implement BF-02 and BF-03.

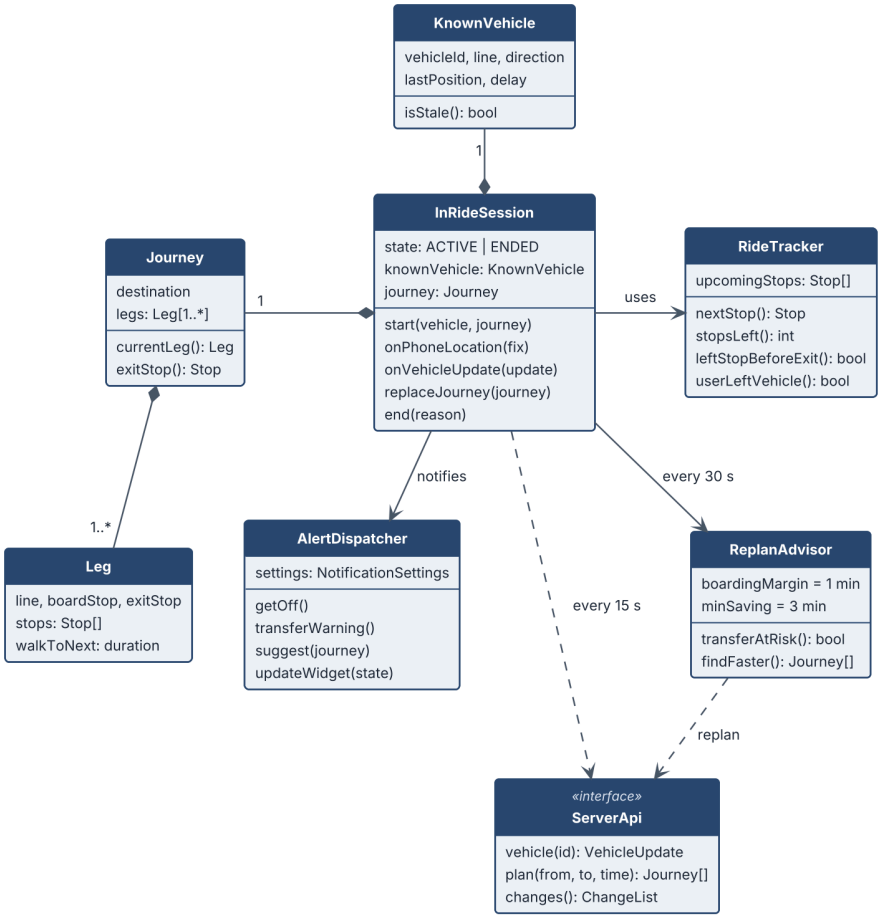


Figure 2. Main classes of the in-ride engine

InRideSession manages active ride state, polling ServerApi for vehicle updates (15s) and evaluating reroutes via ReplanAdvisor (30s) while RideTracker matches 1 Hz phone GPS fixes against route stops. ServerApi is abstracted as an interface to allow replay testing with recorded GPS traces.

5.3 Class diagram: server

Figure 3 shows the main classes of the server modules from Figure 1, and Figure 4 shows the Could have modules. Each class is the public interface of one module, and other modules call it only through these methods. The API Gateway is not shown because it only checks sign-in tokens, forwards requests and limits requests rates. Classes marked «interface» wrap external systems, so each provider can be replaced or mocked in tests. Types such as VehicleState and JourneyOption are defined in section 7.

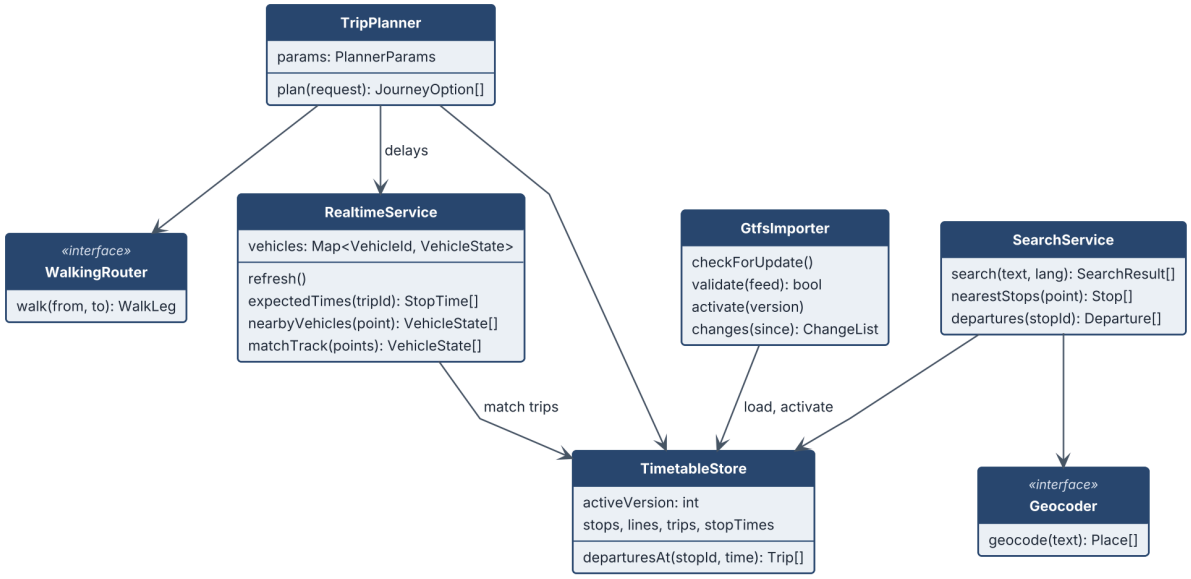


Figure 3. Main classes of the server

TimetableStore gives read-only access to the active GTFS version, which GtfsImporter.activate() replaces in one step. RealtimeService keeps the VehicleState of each vehicle in memory and refreshes it every 10 seconds. TripPlanner.plan() applies the rules in section 5.4.

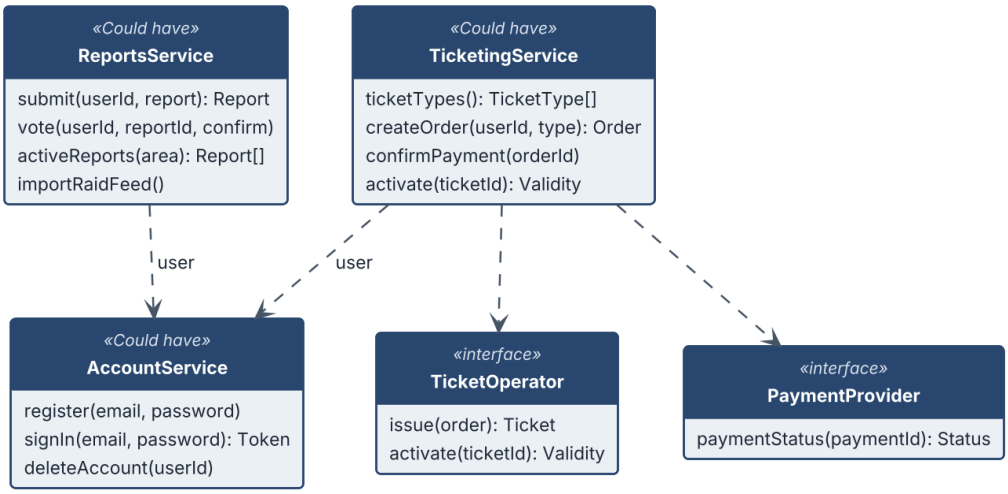


Figure 4. Could have classes of the server

AccountService identifies the user for ReportsService and TicketingService. ReportsService applies the report limit and expiry times from section 4.5.3, and importRaidFeed() runs on a schedule. TicketingService creates the order, checks the payment with PaymentProvider and asks TicketOperator to issue and activate the ticket (BF-06).

5.4 Journey planning

The Trip Planner calculates journey options on the server. It is used in BF-01 and, during in-ride assistance, in BF-03. The implementing team may use an existing open-source planner, such as OpenTripPlanner, or implement a timetable-based algorithm, such as RAPTOR, as long as the rules below are met.

Inputs:

- Starting point: coordinates, or the known vehicle (vehicleId and tripId) during in-ride assistance; the planner takes the vehicle's next stop from the Realtime Service.
- Destination: coordinates.
- Time: a departure time, or the latest arrival time (S-5).
- Step-free option (C-8).

Rules:

1. The planner uses the active GTFS version (section 5.1) and only services that run on the requested date.
2. Where the Realtime Service has an expected time for a trip, the planner uses it instead of the scheduled time. Otherwise it uses the scheduled time and marks that part of the journey as scheduled.
3. When the starting point is a known vehicle, the first part of every option is the rest of the ride on that vehicle. The user can get off at any of its remaining stops.
4. A transfer is allowed only if the walking time between the two stops plus the boarding margin (section 4.2.3) is no longer than the time between arrival and the next departure.
5. Walking times come from the Walking Router. Each walking part, from the starting point, between stops or to the destination, is limited to the maximum walking distance in Table 1.
6. Options are filtered using Pareto dominance on arrival time (or departure time when a latest arrival time is set) and transfer count. Ties are broken first by shortest total walking time, then by latest departure.
7. Options are returned sorted by arrival time (M-5). When a latest arrival time is set, they are sorted by departure time, latest first.

Output:

A list of journey options. Each option is a list of legs (walking or vehicle), with stops, times, line, direction and a scheduled or real-time flag for each leg.

Table 1. Journey planning parameters (stored on the server, section 4)

Parameter	Value
Walking speed	4.5 km/h
Maximum walking distance per walking part	1 km
Maximum number of transfers	3
Search window after the departure time	2 hours
Maximum number of options returned	5

6. UI/UX wireframes

6.1 Overview

The low-fidelity wireframes are in Figma (TrafiK — Section 6 UI/UX wireframes). The file has two pages:

- "Section 6 — Wireframes": every screen, grouped by business flow (BF-01 to BF-06), with the departure and line screens as a separate group and arranged left to right in flow order. Each column has the main screen at the top, a short note with its requirement IDs below it, and then the screen's states.
- "Prototype — clickable": the main screens linked into flows that can be clicked through.

6.2 Conventions

- Screens are drawn for Android at 360 × 800 dp, in greys only.
- One accent colour marks the single primary action of a screen.
- Status is written as a text tag (LIVE, SCHEDULED, TRANSFER AT RISK, GET OFF), never shown by colour alone.
- Could have screens (community reports, tickets, sign-in) carry a "COULD HAVE" marker and a darker background.
- "Open:" in a note marks a UI point whose decision is recorded in 6.5.
- Stop names, lines and times are sample data.

6.3 Screens

Home is the only main screen. Its search field leads to journey planning, "I'm travelling on bus No. X" starts in-ride assistance, a nearby stop opens its departures, and the menu holds notification settings and the Could have functions. Some requirements have no screen of their own: M-12 (the language follows the phone), S-4 (nearby stops are listed on Home), S-10 (notification switches under Menu → Settings), C-8 (a step-free switch on the Search screen) and C-1 (a Sign in screen, opened from the menu or when a signed-out user taps Report or Buy).

Flow	Screens	Requirements
BF-01 Journey planning	Home, Search, Journey options, Journey details	M-1 – M-11, S-1 – S-5, S-12
BF-02 Starting in-ride assistance	Select vehicle, Automatic detection	M-14, M-15, S-6, S-7
BF-03 In-ride tracking	Tracking, Transfer at risk	M-15 – M-19, S-8, S-11, S-13
Departures and lines (no business flow)	Stop departures, Line	M-3, M-13, S-9
BF-05 Community reports	Menu, Report type, Confirm, Reports on	C-1 – C-5

(Could have)	map	
BF-06 Tickets *(Could have)*	Tickets, Payment, My tickets, Active ticket	C-1, C-6, C-7

Table 2. Screens and requirements

6.4 Example scenario

A visitor at the Cathedral wants to reach Didlaukio g. 47. The flow can be followed in prototype flow 1, “Plan a journey and ride”.

1. **Home.** She taps the search field.
2. **Search.** The start is already “My location”. She types the destination and picks it from the list.
3. **Journey options.** She compares three options and chooses bus 3G with a transfer to bus 35.
4. **Journey details.** She reads the step-by-step guide, walks to Katedros a., boards 3G and taps “I’m travelling on bus No. X”.
5. **Tracking.** The screen shows the next stop and how many stops are left. She locks her phone and follows the ride on the lock-screen widget.
6. **Transfer at risk.** The bus is 3 minutes late, so the app warns that she may miss bus 35 and offers a new option. She taps “Use selected journey”.
7. **Transfer.** At the transfer stop, the app shows the walk to the stop of her next bus. When that bus arrives, the app proposes it as her current vehicle, she confirms, and tracking continues on the new bus.
8. **Get-off alert.** One stop before her exit, the phone shows “Get off at the next stop”. After the last bus, the app shows the walk to the destination and the ride ends.

The other prototype flows cover automatic detection, departures and lines, and the two Could have flows, community reports and tickets.

6.5 UI decisions

The following UI decisions apply to both the wireframes and the implementation:

1. Home is a single map screen. All other functions are reached from it or from its menu.
2. Time status tags are LIVE and SCHEDULED, and delays are written as “3 min late” (Lithuanian: “vėluoja 3 min.”).
3. A journey option is tagged LIVE only if all its vehicle legs have real-time data; otherwise it is tagged SCHEDULED.
4. The button label is always “I’m travelling on bus No. X”; the line and vehicle are chosen on the Select vehicle screen.
5. Automatic detection asks the user in a dialog with Yes and No buttons.
6. The Line screen shows departure times from the first stop only.
7. Report is opened from the Home menu.

7. Data model

This section defines the business entities of TrafiK. Figure 5 shows timetable and real-time data on the server, Figure 6 shows journeys and the data stored on the phone, and Figure 7 shows the Could have entities for accounts, reports and tickets. The REST API endpoints and service interfaces (section 5.3) use the same entity names, fields and types.

Conventions:

- Types are string, int, float, bool, decimal (amount in euros with two decimal places), UUID, date (YYYY-MM-DD), datetime (ISO 8601 in UTC) and duration (whole seconds). Coordinates (lat, lon) are WGS 84 decimal degrees, and a Point is one lat and lon pair.
- A ? after a type marks an optional field, and [] marks a list. Values separated by | are the only allowed values of that field.
- «device» marks data stored only on the phone. All other entities are stored on the server or, for journeys, only returned by it.
- A field ending in Id holds the id of the entity with that name; for example, lineId is a Line.id.

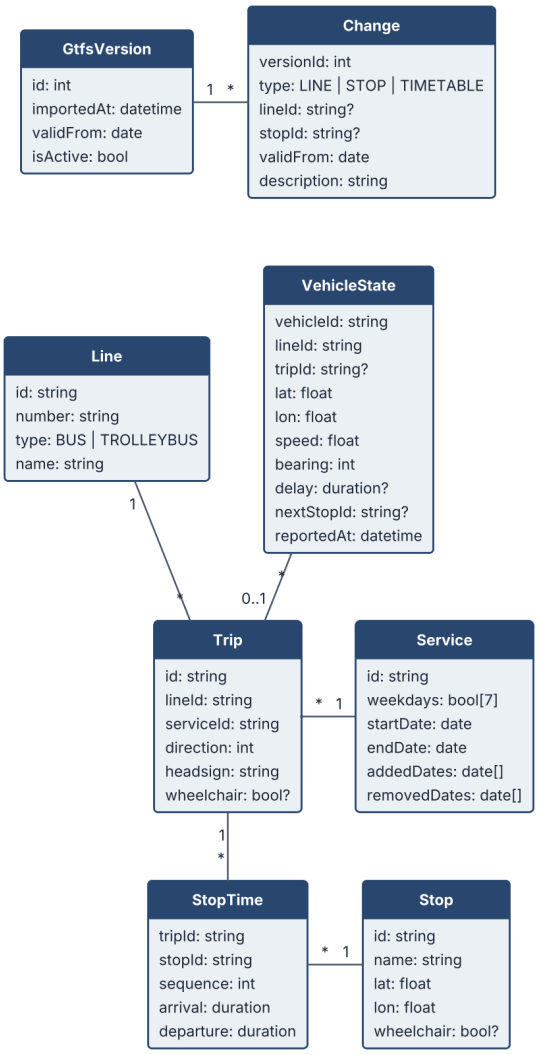


Figure 5. Timetable and real-time data

- Stop, Line, Trip, StopTime and Service are loaded from the GTFS files stops.txt, routes.txt, trips.txt, stop_times.txt, calendar.txt and calendar_dates.txt. Each row belongs to one GtfsVersion, so each table is keyed by versionId and id, and only the active version is read (section 5.1).
- StopTime arrival and departure are measured from midnight of the service day and can exceed 24 hours for trips that run past midnight, as in GTFS.
- VehicleState is kept only in server memory and replaced every 10 seconds (BF-04). tripId and delay are empty when the vehicle cannot be matched to a trip.
- Change lists what a new GtfsVersion changes, and the app uses it for route-change notices (S-9).

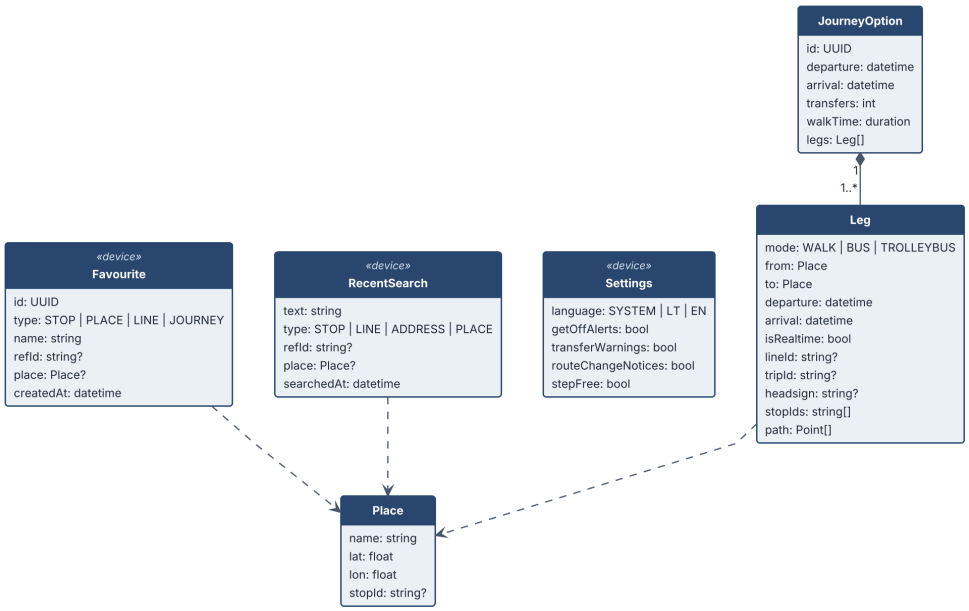


Figure 6. Journeys and data stored on the phone

- JourneyOption and Leg are calculated by the Trip Planner and are not stored on the server (section 4.5.1). When the user selects a JourneyOption, the app maps it to the in-ride domain object Journey (Figure 2), resolving from/to places of vehicle legs into boardStop and exitStop.
- Leg mode is WALK, BUS or TROLLEYBUS. A walking leg has no lineId, tripId or headsign. path is the line drawn on the map for that leg.
- In Favourite and RecentSearch, refId holds the stop or line ID, and place holds the coordinates/stop for addresses, places, and map pins (for a JOURNEY favourite, place holds the destination while the origin defaults to the user's location or a serialized origin ID in refId). The phone keeps the 20 most recent searches.
- Tickets from Figure 7 are also copied to the phone, so the active ticket can be shown offline (C-7).

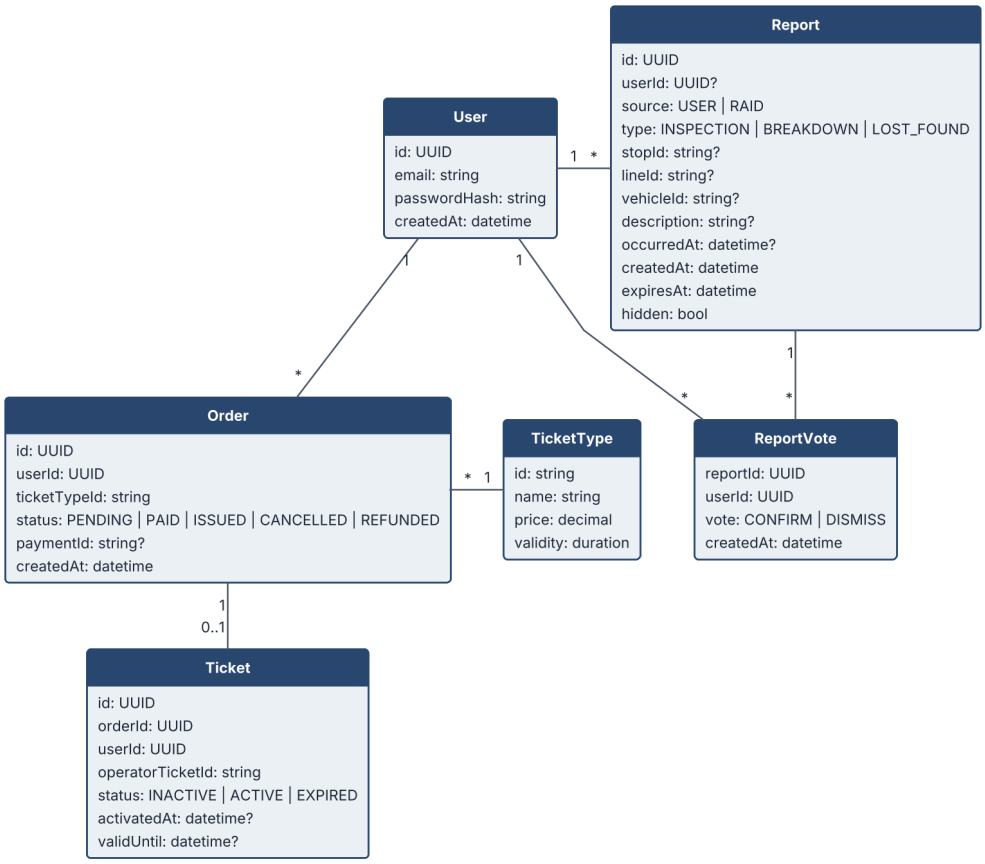


Figure 7. Accounts, community reports and tickets (Could have)

- Report stores a stop, line or vehicle instead of coordinates. The app converts the user's location to the nearest stop, so precise locations are not linked to accounts (section 4.5.1). userId is empty for reports imported from raid.lt. occurredAt (datetime?) is the time a lost or found item was lost or found, as entered by the user (BF-05).
- expiresAt is createdAt plus the time for the report type in section 4.5.3, and hidden is set when the dismissal rule in the same section is met. Each user can vote once per report, and vote is CONFIRM or DISMISS.
- Order status follows BF-06: PENDING, then PAID, then ISSUED, or CANCELLED or REFUNDED if the flow fails. When a ticket is activated, validUntil is set from the operator's response.
- Card data is never stored. paymentId is the Payment Provider's reference for the payment.